



DiabetesRisk

Tamizaje temprano de riesgo de diabetes tipo 2

Entrega #2



Jared Foster Orduz - Jeferson David Vargas Toca
Andres Felipe Florez Garces - Oscar Enrique Morillo

Universidad de los Andes — Curso MAIA

Septiembre de 2026

Enlaces de Proyecto

- **Repositorio (GitHub):** https://github.com/jadfof/Microproyecto_DiabetesRisk
- **MLflow (instancia EC2):** <http://44.204.142.207:5000/>
- **API (FastAPI) :** <http://44.204.142.207:8000/docs>
- **Tablero — Streamlit (instancia EC2):** <http://44.204.142.207:8501/>

1. Problema y contexto

La diabetes tipo 2 y la prediabetes se diagnostican con frecuencia de forma tardía, dado que el diagnóstico confirmatorio requiere exámenes de laboratorio (glucemia en ayunas, hemoglobina glicosilada) que implican costo, tiempo y acceso a un centro de salud. Este proyecto desarrolla un prototipo de tamizaje que estima el riesgo de un individuo usando únicamente variables demográficas y de estilo de vida, para priorizar a quién remitir a exámenes confirmatorios. Sin cambios de fondo respecto a la Entrega 1.

2. Pregunta de negocio

Pregunta de negocio: ¿es posible estimar la probabilidad de que una persona tenga diabetes o prediabetes usando únicamente variables demográficas y de estilo de vida —sin exámenes de laboratorio— con el fin de priorizar a quién remitir a un examen clínico confirmatorio?

3. Alcance del proyecto

- **Datos:** [Diabetes Health Indicators Dataset \(BRFSS 2015, CDC\)](#), con fines estrictamente académicos.
- **Modelo:** clasificación binaria supervisada, empaquetada para servir inferencias.
- **Tablero:** formulario de evaluación individual con predicciones del modelo + panel de estadísticas descriptivas de la población.
- **Despliegue:** contenedores Docker, con orquestación e infraestructura en AWS (en curso; ver sección 7.4).
- **Limitaciones:** prototipo académico, no un dispositivo médico certificado; no sustituye el diagnóstico ni el criterio de un profesional de la salud.

Cambio respecto a la Entrega 1: en esta entrega se avanzó del alcance conceptual (mockup) a un modelo entrenado y un tablero funcional con predicciones reales, ambos desplegados y verificables en la instancia EC2 del proyecto.

4. Descripción del conjunto de datos

Se mantiene el Diabetes Health Indicators Dataset (BRFSS 2015, CDC), 253,680 registros, sin valores nulos, con 86.1% de encuestados sin diabetes y 13.9% con diabetes o prediabetes (desbalance ya identificado en la Entrega 1).

Cambio respecto a la Entrega 1: de las 21 variables originales se seleccionó un subconjunto final de 11 predictores —HighBP, HighChol, BMI, Smoker, Stroke, HeartDiseaseorAttack, PhysActivity, GenHlth, DiffWalk, Age e Income—, priorizando las variables con mayor correlación (positiva o negativa) con la variable objetivo, y descartando variables redundantes o de bajo aporte, para simplificar el formulario del tablero sin perder poder predictivo. El detalle de esta selección se retoma en la sección 5.1.2.

5. Modelamiento

5.1 Modelos entrenados individualmente

5.1.1 Entrenamiento de los modelos

Se entrenaron y compararon 4 modelos, todos con partición estratificada 80/20 (train/test, random_state=42) para preservar la proporción de clases en ambos conjuntos:

- **Regresión Logística (base):** sin tratar el desbalance de clases, como línea base de referencia. Variables estandarizadas con StandardScaler.
- **Regresión Logística + SMOTE:** mismas variables, con sobremuestreo (SMOTE) de la clase minoritaria antes de entrenar.
- **Random Forest (class_weight balanceado):** 200 árboles, profundidad máxima 10, penalizando más los errores sobre la clase minoritaria.
- **Random Forest + SMOTE:** mismos hiperparámetros del árbol, combinado con sobremuestreo.

SMOTE (Synthetic Minority Over-sampling Technique) genera ejemplos sintéticos de la clase minoritaria únicamente sobre el conjunto de entrenamiento, nunca sobre el conjunto de prueba, para evitar fuga de información. class_weight='balanced' logra un efecto similar sin modificar los datos, penalizando más los errores sobre la clase minoritaria directamente en la función de pérdida.

5.1.2 Selección de características

Las 11 variables predictoras se seleccionaron a partir del análisis de correlación de Pearson realizado en la Entrega 1, priorizando las variables con mayor correlación (absoluta) con Diabetes_binary y descartando variables redundantes, de baja señal o difíciles de auto-reportar en un formulario (por ejemplo, se excluyeron CholCheck, AnyHealthcare, NoDocbcCost y Education por su correlación cercana a cero o su naturaleza administrativa más que clínica).

Correlación de variables con Diabetes_binary (resaltadas: seleccionadas)

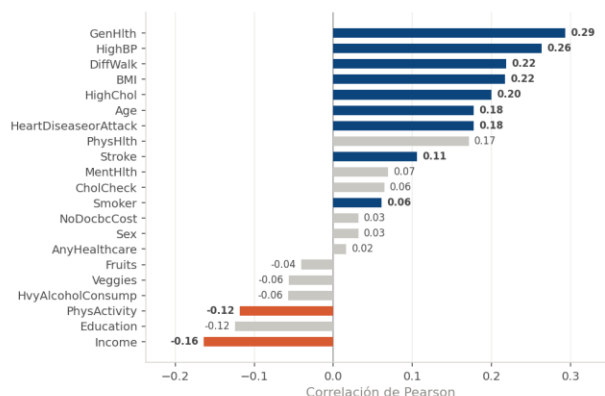


Figura 1. Correlación de cada variable con Diabetes_binary. En azul/naranja, las 11 variables seleccionadas como predictoras finales; en gris, las variables descartadas.

5.1.3 Evaluación de los modelos

Modelo	Accuracy	Precision	Recall	F1-score	ROC-AUC
Regresión Logística (base)	0.861	0.511	0.148	0.229	0.814
Regresión Logística + SMOTE	0.726	0.306	0.757	0.435	0.813
Random Forest (class_weight)	0.721	0.303	0.772	0.436	0.819
Random Forest + SMOTE	0.788	0.352	0.617	0.448	0.812

Comparación de modelos: DiabetesRisk

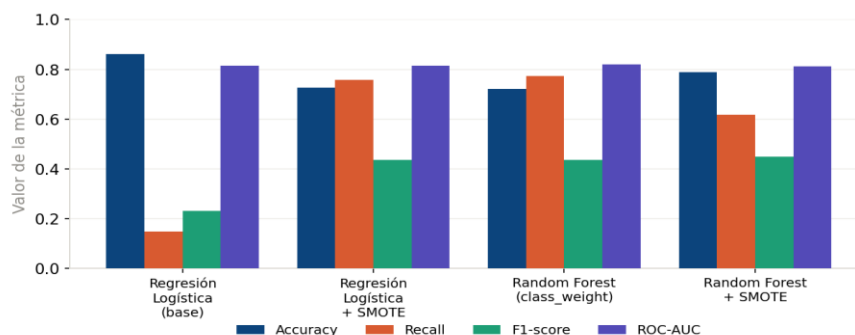


Figura 2. Comparación de métricas entre los 4 modelos entrenados.

Matriz de confusión del modelo seleccionado (random_forest_balanced), sobre el conjunto de prueba (50,736 registros):

	Predicho: Sin diabetes	Predicho: Con diabetes/prediab.
Real: Sin diabetes	31,155 (verdaderos negativos)	12,512 (falsos positivos)
Real: Con diabetes/prediab.	1,607 (falsos negativos)	5,462 (verdaderos positivos)

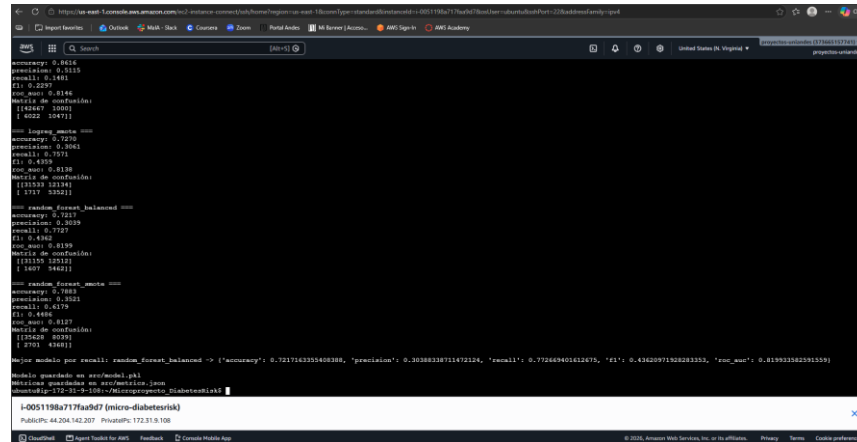
Interpretación: de los 7,069 casos reales de diabetes/prediabetes, el modelo detecta correctamente 5,462 (77.3%, verdaderos positivos) y deja pasar 1,607 (22.7%, falsos negativos). A cambio, remite a examen confirmatorio a 12,512 personas que en realidad no tienen la condición (falsos positivos, 28.6% de los negativos reales) — balance intencional para un caso de uso de tamizaje, donde el costo de un examen adicional es mucho menor que el costo de no detectar a tiempo un caso real.

5.2 Gestión de experimentos con MLflow

Todos los experimentos se registraron en un servidor MLflow desplegado en la instancia EC2 del proyecto, accesible en <http://44.204.142.207:5000/>

5.2.1 Entrenamiento de los modelos

Cada corrida (run) queda registrada con su tipo de modelo (model_type), sus hiperparámetros completos (n_estimators, max_depth, etc., vía model.get_params()), y si se aplicó SMOTE o no (smote=True/False), permitiendo reproducir exactamente cómo se entrenó cada modelo.



```
accuracy: 0.8624
precision: 0.5113
recall: 0.1481
f1: 0.2297
roc_auc: 0.8184
Matriz de confusión:
[[1569 1001]
 [ 802 1047]]

=== logit_smote ===
accuracy: 0.7939
precision: 0.1061
recall: 0.3971
f1: 0.4305
roc_auc: 0.8138
Matriz de confusión:
[[10109 12191]
 [ 1717  5152]]

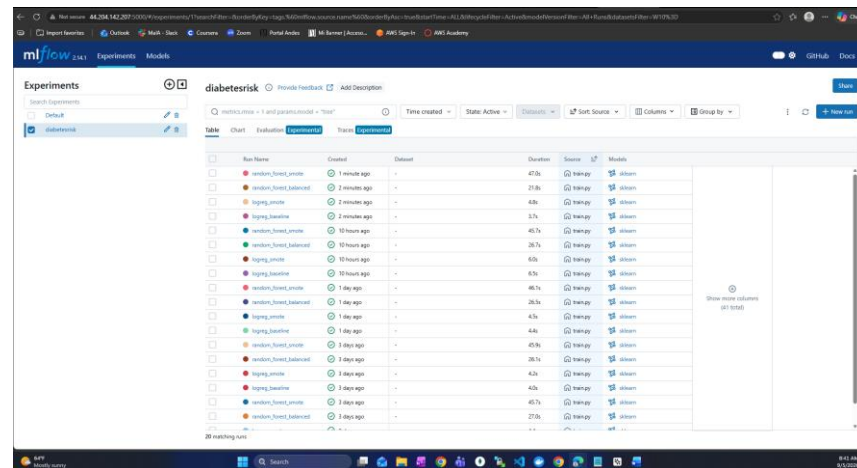
=== random_forest_balanced ===
accuracy: 0.7217
precision: 0.2629
recall: 0.7727
f1: 0.4182
roc_auc: 0.8199
Matriz de confusión:
[[11108 12012]
 [ 1007  3401]]

=== random_forest_smote ===
accuracy: 0.7883
precision: 0.1081
recall: 0.4179
f1: 0.4485
roc_auc: 0.8127
Matriz de confusión:
[[15628 8093]
 [ 2793  4380]]

mejor modelo por recall: random_forest_balanced -> {"accuracy": 0.721743355408386, "precision": 0.30388328711472124, "recall": 0.772684905612675, "f1": 0.43620971508283353, "roc_auc": 0.81993182591559}

Modelo guardado en src/model.pkl
Metilación generada de microdiabetes_risk:
sha256ip:172-31-5-1181-Microdiabetes_risktest.pkl
```

Figura 3. Terminal de la instancia EC2 (usuario ubuntu, IP pública 44.204.142.207 visible en el panel inferior) mostrando el entrenamiento y las métricas de las 4 corridas.



Run Name	Created	Duration	Source	Models
random_forest_smote	1 minute ago	47.0s	train.py	sklearn
random_forest_balanced	2 minutes ago	21.8s	train.py	sklearn
logit_smote	2 minutes ago	6.8s	train.py	sklearn
logit_balanced	2 minutes ago	5.7s	train.py	sklearn
random_forest_smote	10 hours ago	45.7s	train.py	sklearn
random_forest_balanced	10 hours ago	26.7s	train.py	sklearn
logit_smote	10 hours ago	6.8s	train.py	sklearn
logit_balanced	10 hours ago	6.7s	train.py	sklearn
random_forest_smote	1 day ago	46.7s	train.py	sklearn
random_forest_balanced	1 day ago	26.7s	train.py	sklearn
logit_smote	1 day ago	6.7s	train.py	sklearn
logit_balanced	1 day ago	4.4s	train.py	sklearn
random_forest_smote	3 days ago	45.6s	train.py	sklearn
random_forest_balanced	3 days ago	26.7s	train.py	sklearn
logit_smote	3 days ago	6.7s	train.py	sklearn
logit_balanced	3 days ago	4.5s	train.py	sklearn
random_forest_smote	3 days ago	45.7s	train.py	sklearn
random_forest_balanced	3 days ago	27.0s	train.py	sklearn

Figura 4. Interfaz de MLflow accedida desde el navegador con la IP pública de la instancia EC2 (44.204.142.207:5000) en la URL, mostrando las 4 corridas del experimento diabetesrisk.

5.2.2 Selección de características

A partir de esta entrega, el listado de variables usadas (features) y su cantidad (n_features) se registran explícitamente como parámetros de cada corrida en MLflow, además de quedar fijos en el código versionado (src/train.py), de modo que cualquier corrida futura sea trazable respecto a qué variables se usaron.

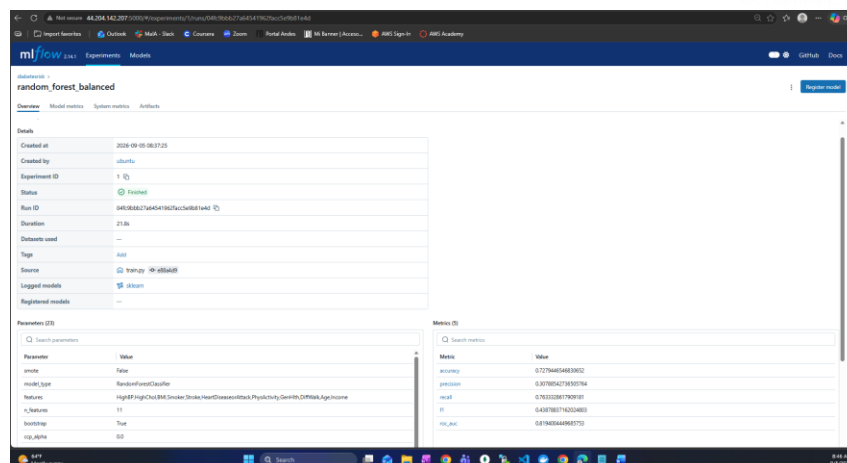


Figura 5. Vista de la corrida `random_forest_balanced` en MLflow: los 23 parámetros registrados incluyen features (listado completo de las 11 variables) y `n_features=11`, junto con las 5 métricas de evaluación.

5.2.3 Evaluación de los modelos

Las métricas de evaluación (`accuracy`, `precision`, `recall`, `f1`, `roc_auc`) se registran automáticamente por corrida y pueden compararse visualmente desde la pestaña `Chart` de MLflow, sin necesidad de exportar los datos manualmente.

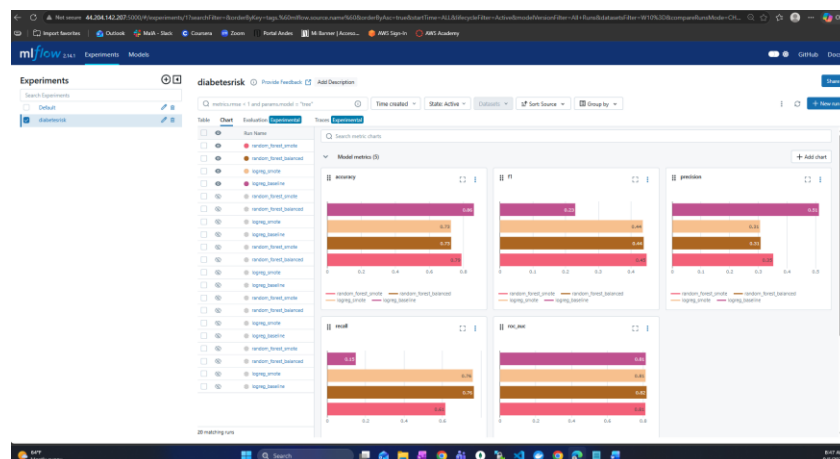


Figura 6. Vista de comparación de métricas de MLflow (`accuracy`, `f1`, `precision`, `recall`, `roc_auc`) para las 4 corridas registradas, generada directamente desde la interfaz web en la instancia EC2.

5.3 Análisis extendido del modelo seleccionado

Para profundizar en el comportamiento del modelo `random_forest_balanced` más allá de las métricas puntuales, se incluye a continuación su configuración completa de hiperparámetros (extraída directamente de MLflow), su curva ROC, y la importancia de cada variable según el propio modelo.

5.3.1 Hiperparámetros del modelo final

Hiperparámetro	Valor
<code>n_estimators</code>	200
<code>max_depth</code>	10
<code>criterion</code>	gini
<code>max_features</code>	sqrt
<code>min_samples_split</code>	2
<code>min_samples_leaf</code>	1

bootstrap	True
class_weight	balanced

5.3.2 Curva ROC

La curva ROC confirma el ROC-AUC de 0.819 reportado por MLflow: el modelo discrimina considerablemente mejor que un clasificador aleatorio (línea diagonal) en todo el rango de umbrales de decisión posibles, lo cual es deseable ya que el umbral específico (actualmente 0.5 por defecto) podrá calibrarse en semanas posteriores sin necesidad de reentrenar el modelo.

Curva ROC — random_forest_balanced

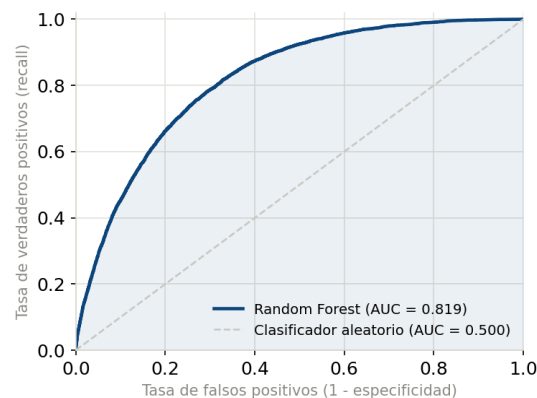


Figura 7. Curva ROC del modelo random_forest_balanced sobre el conjunto de prueba (AUC = 0.819).

5.3.3 Importancia de variables

La importancia de variables calculada directamente por el Random Forest (basada en la reducción de impureza Gini) confirma y refina el análisis de correlación de la sección 5.1.2: salud general percibida (GenHlth), presión arterial alta (HighBP) e IMC (BMI) concentran más del 65% de la importancia total del modelo, mientras que Smoker, Stroke y PhysActivity —aunque se mantuvieron en el modelo por su relevancia clínica y su correlación individual— aportan comparativamente poco poder predictivo adicional una vez consideradas las demás variables.

Importancia de variables — random_forest_balanced

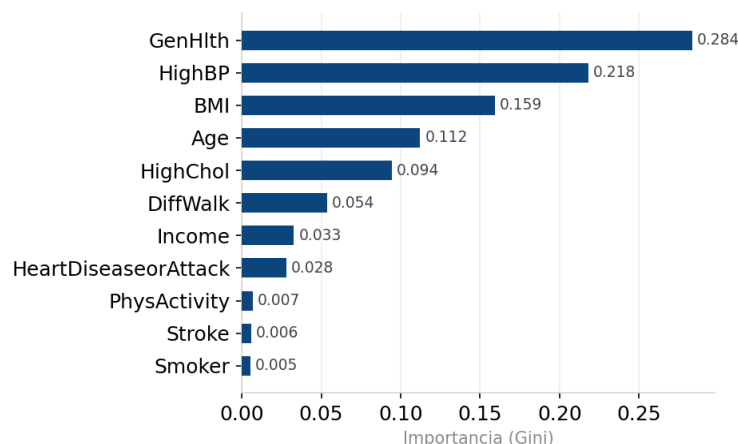


Figura 8. Importancia de cada variable en el modelo `random_forest_balanced`, medida como reducción promedio de impureza Gini.

6. Conclusiones de los modelos

- El modelo base (sin tratar el desbalance) alcanza una accuracy engañosamente alta (86.2%) pero un recall muy bajo (14.8%): el modelo aprende a predecir casi siempre "sin diabetes", que es la clase mayoritaria. Esto confirma la importancia de no usar accuracy como única métrica en problemas desbalanceados.
- Tanto SMOTE como `class_weight='balanced'` mejoran sustancialmente el recall frente al modelo base, a costa de una caída esperada en precision y accuracy — el trade-off correcto para un caso de uso de tamizaje, donde priorizar sensibilidad (no dejar pasar casos de riesgo) es más importante que la precisión exacta. Sin embargo, las dos técnicas no son intercambiables: comparadas sobre el mismo modelo (Random Forest), `class_weight` logró un recall notablemente mayor que SMOTE (0.772 vs 0.617), lo que sugiere que penalizar directamente la función de pérdida fue más efectivo que el sobremuestreo sintético en este caso.
- El modelo final seleccionado (Random Forest con `class_weight='balanced'`) se eligió priorizando el recall como criterio de selección, no el F1-score ni la accuracy — de hecho, Random Forest + SMOTE obtuvo un F1 ligeramente mayor (0.448 vs 0.436) y mejor accuracy/precision que el modelo elegido. Esta decisión es consistente con el enfoque de tamizaje del proyecto: en este contexto, es preferible un modelo que detecte más casos reales de riesgo (mayor recall) aunque genere más falsos positivos, antes que uno que optimice el balance general entre precisión y cobertura.
- El ROC-AUC se mantiene estable entre 0.812 y 0.819 en los 4 modelos, lo que indica que la capacidad de discriminación subyacente del modelo cambia poco; lo que sí cambia notablemente es el punto de operación (umbral) resultante de cada estrategia de balanceo.
- Las variables más influyentes son consistentes con los hallazgos de correlación: salud general percibida (GenHlth), presión arterial alta (HighBP) e IMC (BMI) concentran la mayor parte del poder predictivo.
- **Próximos pasos:** calibrar el umbral de decisión del modelo seleccionado, y adicionalmente validar la calibración de las probabilidades que produce (`predict_proba`) — un Random Forest no garantiza que un valor como 35% refleje una frecuencia real observada, lo cual es distinto de solo ajustar el punto de corte de decisión; esto es relevante porque el tablero muestra ese porcentaje directamente al usuario. También se debe probar validación cruzada (en vez de una sola partición train/test) y explorar Gradient Boosting como alternativa adicional antes de empaquetar la versión final del modelo.

7. Tablero desarrollado

El tablero es accesible en <http://44.204.142.207:8501/>

7.1 Empaquetamiento del modelo

Al finalizar el entrenamiento, el script `mlflow/train.py` selecciona automáticamente el modelo con mejor recall entre las 4 corridas (`random_forest_balanced`) y lo serializa con `joblib` en `mlflow/model.pkl`, junto con el listado de features y el nombre del modelo, en un único artefacto:

{model, features, model_name}. Esto permite que el modelo se cargue correctamente sin necesidad de conocer de antemano cuál fue el ganador.

La confirmación de este empaquetamiento es visible en la Figura 3 (línea final: "Modelo guardado en mlflow/model.pkl (modelo seleccionado: random_forest_balanced)").

7.2 API de inferencia

Se desarrolló una API REST con FastAPI (api/main.py) que carga el artefacto empaquetado y expone los siguientes endpoints:

- **/health**: estado de la API, usado por el tablero para verificar conectividad antes de renderizar.
- **/predict**: recibe las 11 variables del formulario y devuelve la probabilidad de riesgo (risk_probability) y su categoría (risk_level).
- **/metrics** y **/model-info**: exponen las métricas de evaluación y los hiperparámetros del modelo seleccionado, para que el tablero los muestre sin tenerlos escritos a mano.

7.3 Implementación del tablero

Se desarrolló el tablero en Streamlit siguiendo la maqueta diseñada en la Entrega 1. **A diferencia de un prototipo inicial que cargaba el modelo directamente, el tablero actual consume el modelo exclusivamente a través de la API** descrita en 7.2: llama a /predict para obtener el riesgo, y a /metrics y /model-info para poblar la página de detalle del modelamiento — sin acceder al archivo .pkl ni recalculando nada localmente.

- **Formulario de evaluación individual**: recoge las 11 variables usadas por el modelo.
- **Predicción en tiempo real vía API**: al enviar el formulario, el tablero hace un POST a /predict y muestra el porcentaje de riesgo, clasificado en tres niveles (bajo <20%, moderado 20–45%, alto >45%) con su color correspondiente, una barra de progreso y una comparación contra la prevalencia promedio de la población.
- **Panel de estadísticas descriptivas**: número de registros analizados, prevalencia general, nombre del modelo activo, y dos gráficas de prevalencia por IMC y por grupo de edad, calculadas en vivo sobre el dataset completo, cada una con una explicación breve de qué variable resalta y por qué.
- **Mensaje de responsabilidad**: se aclara que el resultado es una estimación de un prototipo académico, no un diagnóstico médico.
- **Aislamiento de dependencias**: tablero, API y MLflow corren en tres entornos virtuales de Python separados en la misma instancia EC2, evitando conflictos de versiones (p. ej. protobuf entre Streamlit y MLflow) — un paso intermedio antes de la contenedorización con Docker.

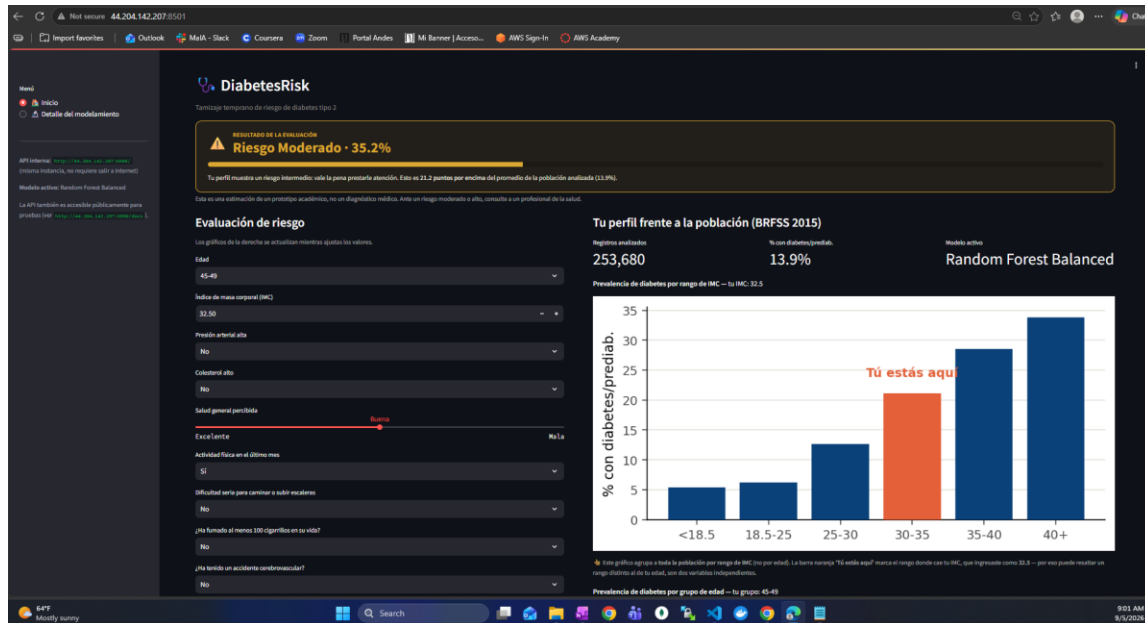


Figura 9. Tablero DiabetesRisk en funcionamiento, accedido desde la IP pública de la instancia EC2 (44.204.142.207:8501).

Relación con la pregunta de negocio: el resultado individual (panel izquierdo) responde directamente a la pregunta de negocio —estimar el riesgo usando solo variables fáciles de reportar—, obtenido en vivo desde el modelo servido por la API; el panel de estadísticas le da al usuario contexto poblacional para interpretar su resultado.

7.4 Estado actual y proyecciones

Estado actual: modelo entrenado, empaquetado y servido a través de una API REST (FastAPI); tablero (Streamlit) que consume esa API para todas sus predicciones y estadísticas; MLflow, API y tablero corriendo de forma persistente en la instancia EC2, cada uno en su propio entorno de Python.

Pendiente para las próximas semanas:

- Empaquetar modelo, API y tablero en contenedores Docker independientes (carpeta docker/, ya definida desde la Entrega 1), replicando en contenedores la misma separación de dependencias lograda con los entornos virtuales de esta entrega.
- Desplegar la versión contenedorizada en la instancia EC2 (o un servicio gestionado equivalente), con los tres componentes orquestados vía docker-compose.

8. Reporte de trabajo en equipo

Para esta segunda entrega, las aportaciones del equipo se distribuyeron de la siguiente manera:

- Infraestructura EC2, configuración y despliegue del servidor MLflow, script de entrenamiento, desarrollo del tablero y registro de experimentos (Jared Foster Orduz).
- Redacción de observaciones y conclusiones sobre el comportamiento de los modelos (Jeferson David Vargas Toca).
- Selección final de variables predictoras y revisión de consistencia con la exploración de datos de la Entrega 1 (Andres Felipe Florez Garces).
- Evaluación comparativa de modelos y revisión funcional del tablero (Oscar Enrique Morillo).